
Project Name: EmotionFlix

Class: Senior Project **Student Name:** Prashant Shah **Email:** pshah7@ramapo.edu

Semester: Spring 2025

Table of Contents

1. [Project Overview](#)
 - [1.1 Purpose and Scope](#)
 - [1.2 Core Implementation Features](#)
2. [System Architecture & Design](#)
 - [2.1 Design Summary](#)
 - [2.2 Technical Stack](#)
 - [2.3 Key Architectural Decisions](#)
3. [Data Structures and File Organization](#)
 - [3.1 Database Architecture & Organization](#)
 - [3.2 TypeScript Data Models](#)
 - [3.3 File Structure and Organization](#)
4. [Classes and Models Implementation](#)
 - [4.1 Service Classes](#)
 - [4.2 Data Models](#)
5. [Core Features and Implementation](#)
 - [5.1 Emotion Detection System](#)
 - [5.2 Recommendation Engine](#)
 - [5.3 User Interface Design](#)
 - [5.4 Data Management](#)
6. [Testing Strategy and Implementation](#)
 - [6.1 Testing Philosophy](#)
 - [6.2 Testing Infrastructure](#)
 - [6.3 Test Coverage Analysis](#)
 - [6.4 Test Execution Commands](#)
7. [Project Setup & Running](#)
 - [7.1 Prerequisites](#)
 - [7.2 Quick Start](#)
 - [7.3 Project Structure Overview](#)
8. [Data Flow and Algorithm](#)
 - [8.1 System Architecture Overview](#)
 - [8.2 Emotion Detection and Processing](#)
 - [8.3 Personalized Emotion Mapping System](#)
 - [8.4 Recommendation Engine Architecture](#)
 - [8.5 Database Operations and Data Flow](#)

9. Conclusion

- 9.1 Implementation Summary
 - 9.2 Developer Experience and Learning Outcomes
 - 9.3 Project Assessment
-

Document Summary

This specifications document provides a comprehensive technical outline of EmotionFlix, an emotion-based movie recommendation application. The document covers the complete implementation from database design to user interface, focusing on what was built and how it functions.

What's Included:

- **System Architecture & Design:** Three-tier architecture with React frontend, Node.js backend, and PostgreSQL database
- **Technical Implementation:** Detailed service layer descriptions, database schema, and component architecture
- **Core Features:** Multi-modal emotion detection using face-api.js, personalized recommendation engine, and user management
- **Data Models & API Design:** TypeScript interfaces, database relationships, and service integrations
- **Testing Infrastructure:** 69 comprehensive tests across frontend and backend with 100% pass rate
- **Algorithm Design:** Emotion-to-genre mapping system with personalized learning capabilities
- **Development Experience:** Technical challenges, solutions implemented, and lessons learned

1. Project Overview

1.1 Purpose and Scope

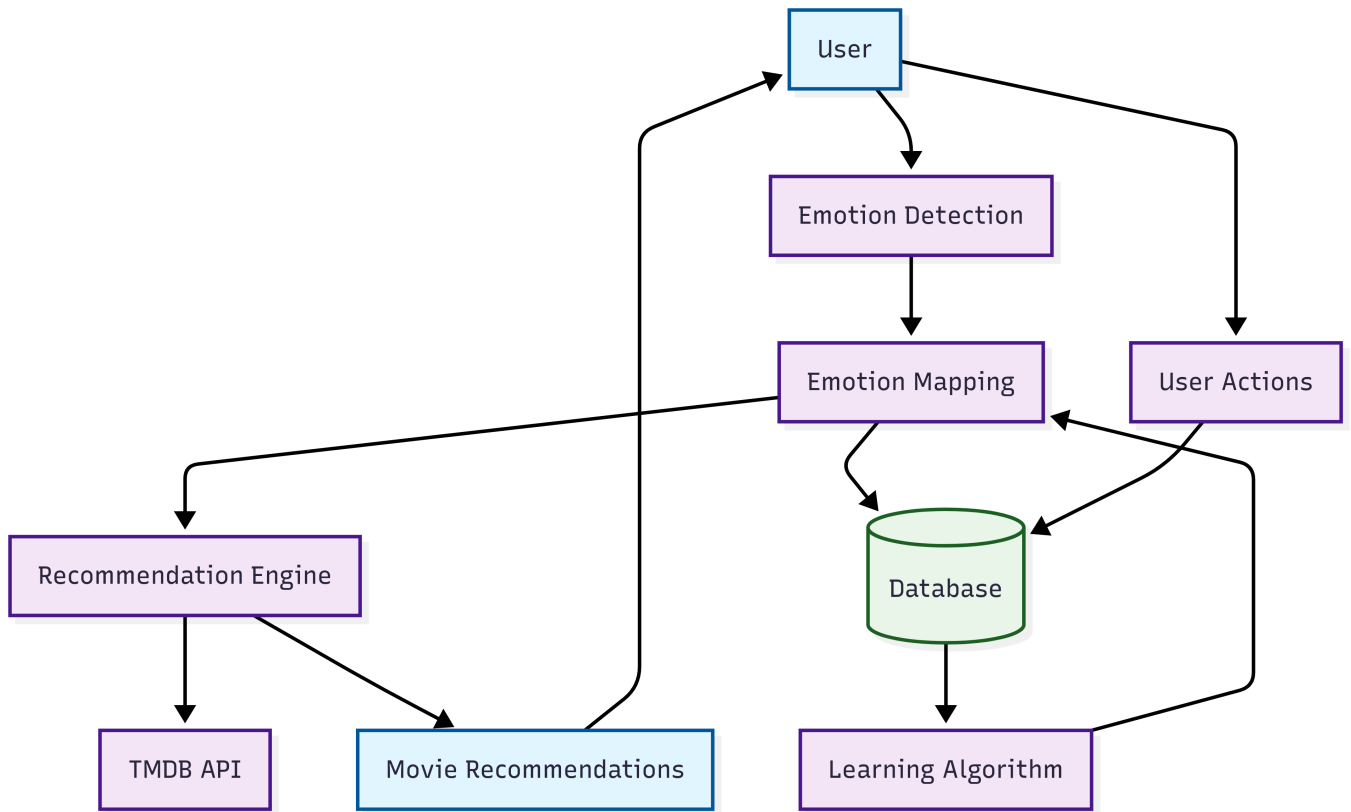
EmotionFlix addresses the fundamental challenge of movie discovery by introducing emotion-driven recommendations. Traditional recommendation systems rely on genre preferences and ratings, but EmotionFlix analyzes users' current emotional states to suggest movies that align with their immediate emotional needs.

1.2 Core Implementation Features

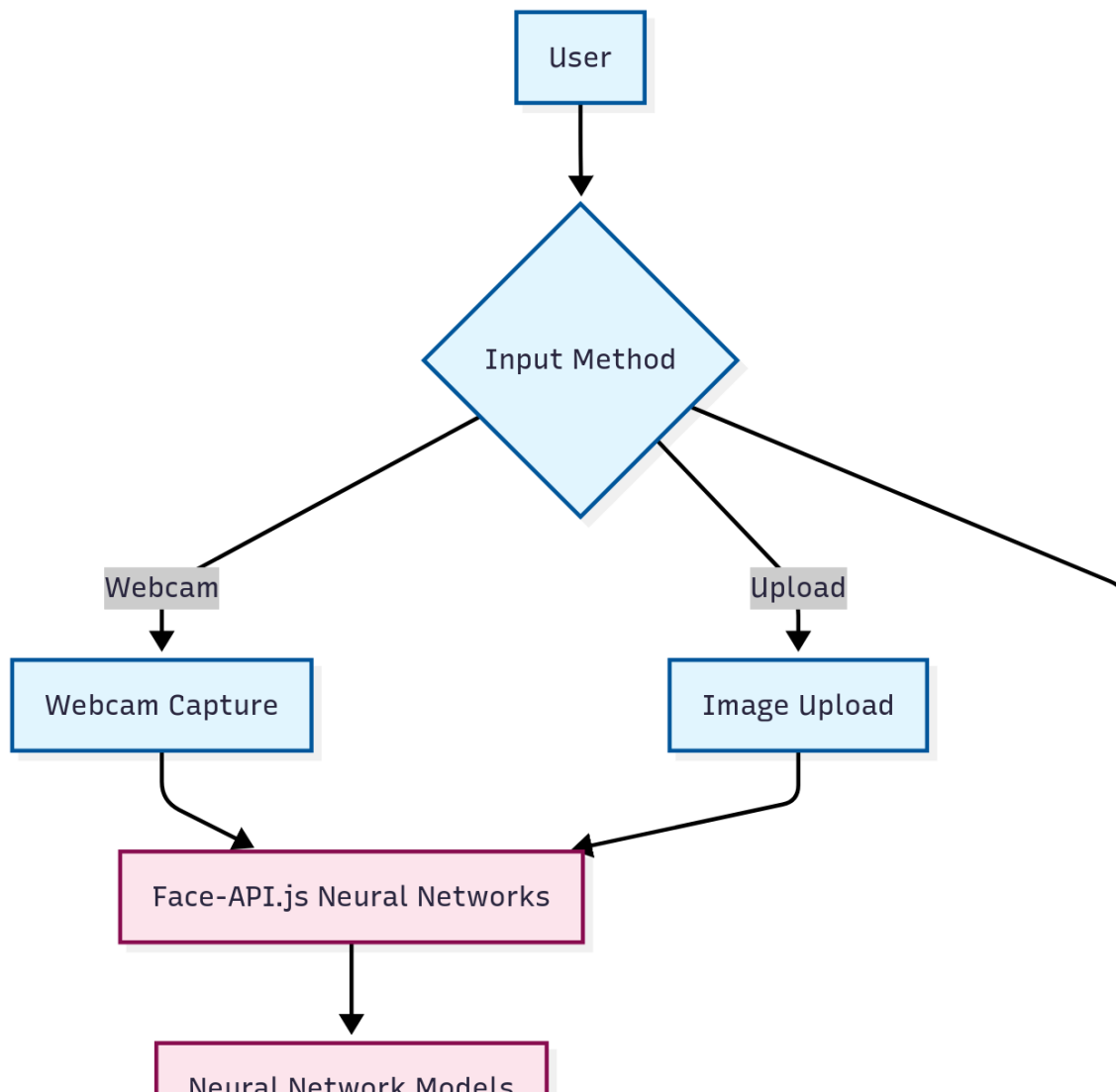
- **Multi-Modal Emotion Detection:** Real-time facial analysis using face-api.js neural networks with webcam capture, photo upload, and manual input options
- **Personalized Recommendation Algorithm:** Hybrid approach combining emotion-to-genre mapping, user emotion learning, and TMDB movie metadata
- **Privacy-First Architecture:** All emotion processing happens client-side with no image data stored
- **Full-Stack Development:** Complete application with responsive React frontend, Express.js API, and PostgreSQL database

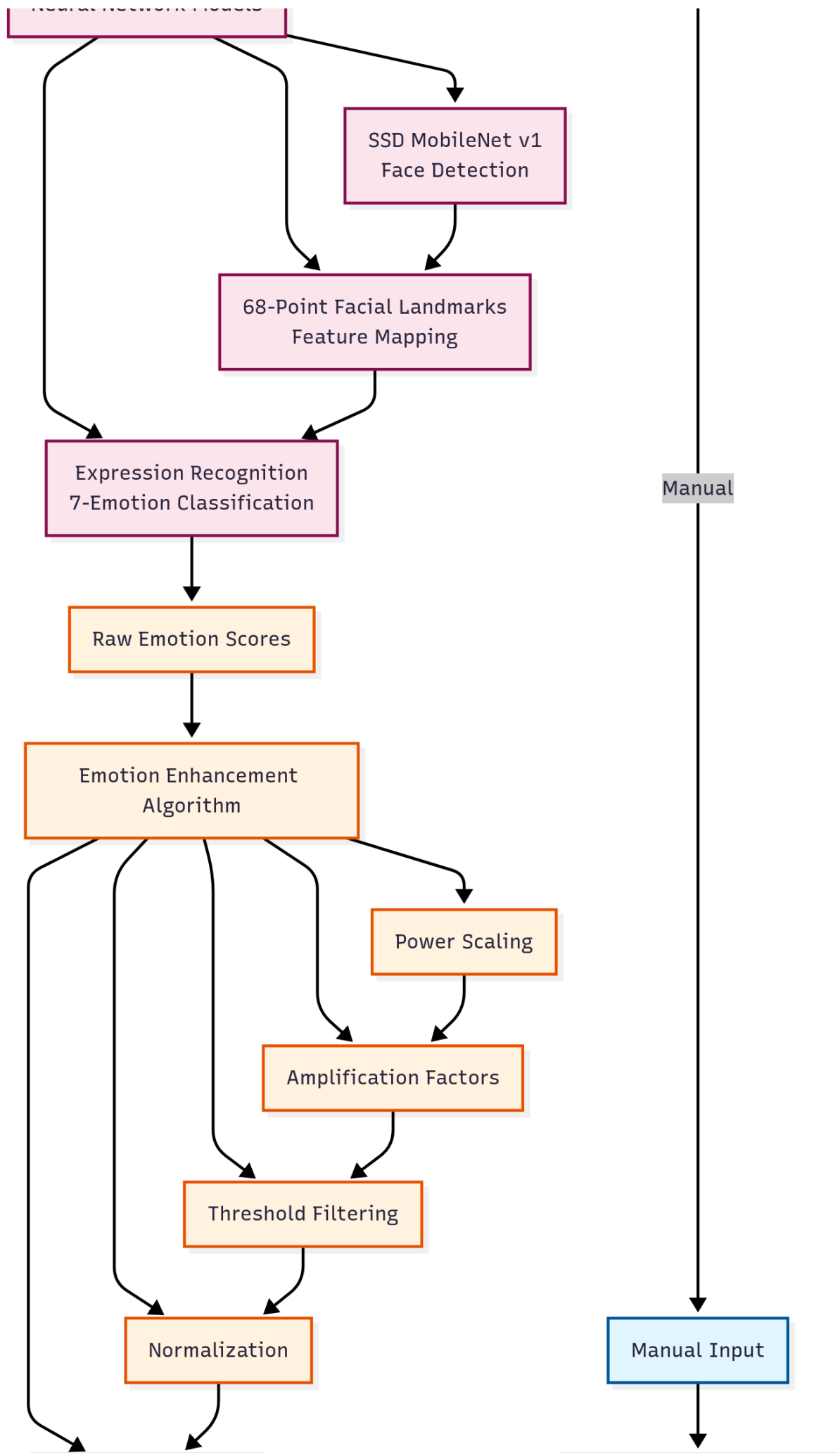
2. System Architecture & Design

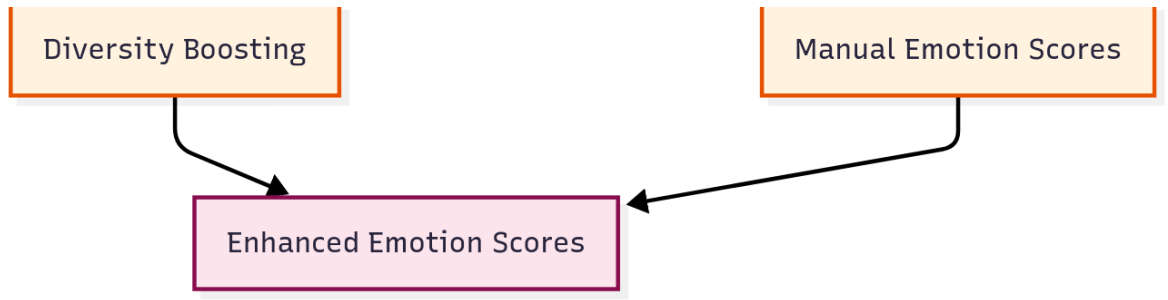
Application Overview Data Flow Diagram (DFD)



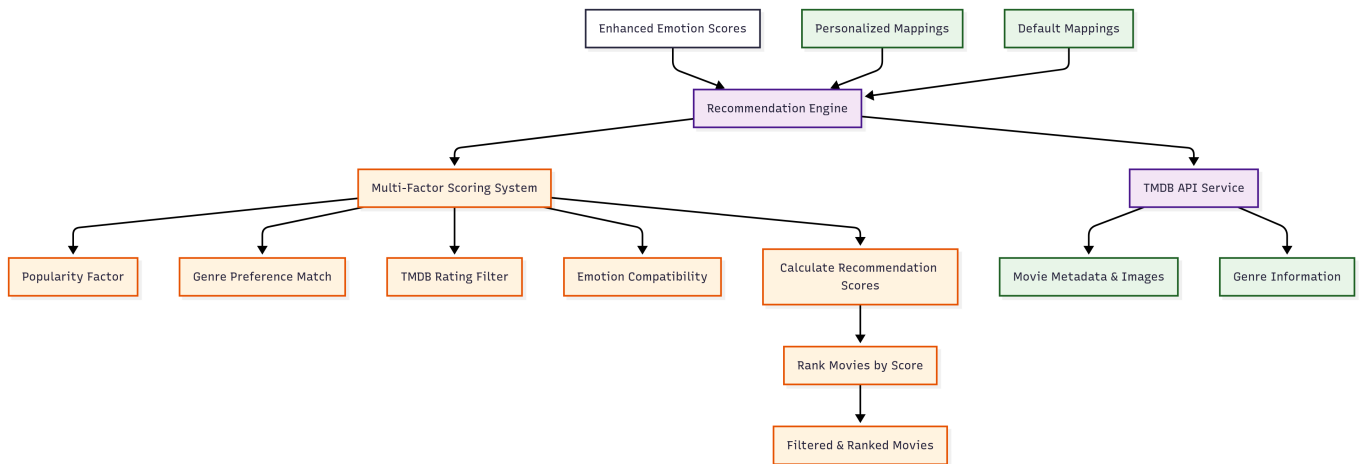
Emotion Detection DFD



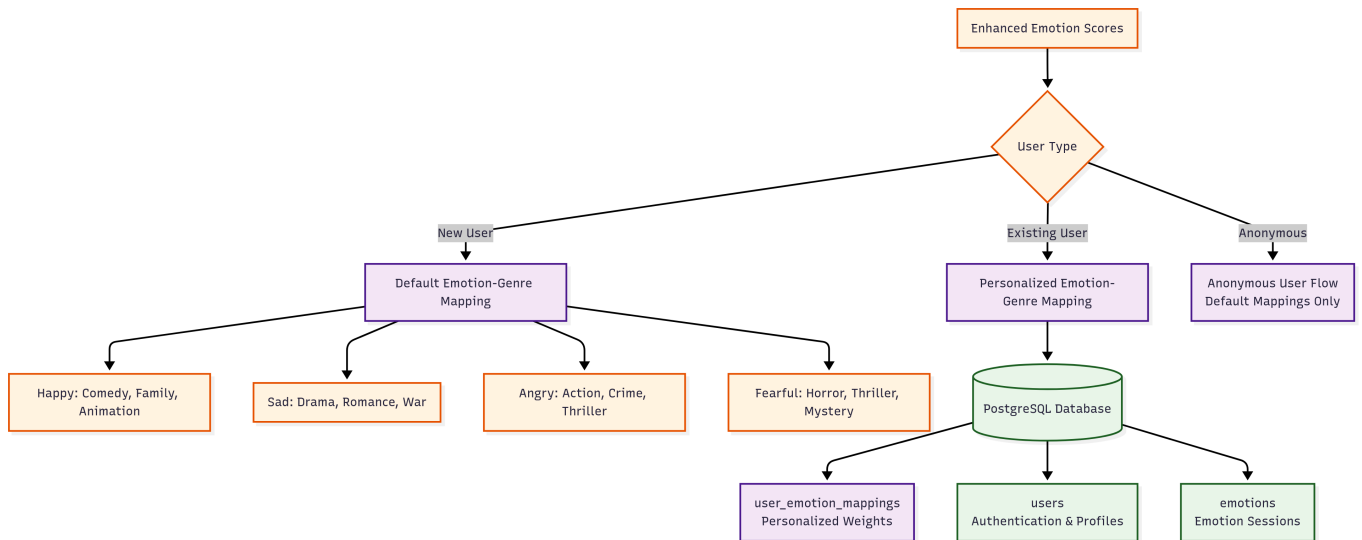




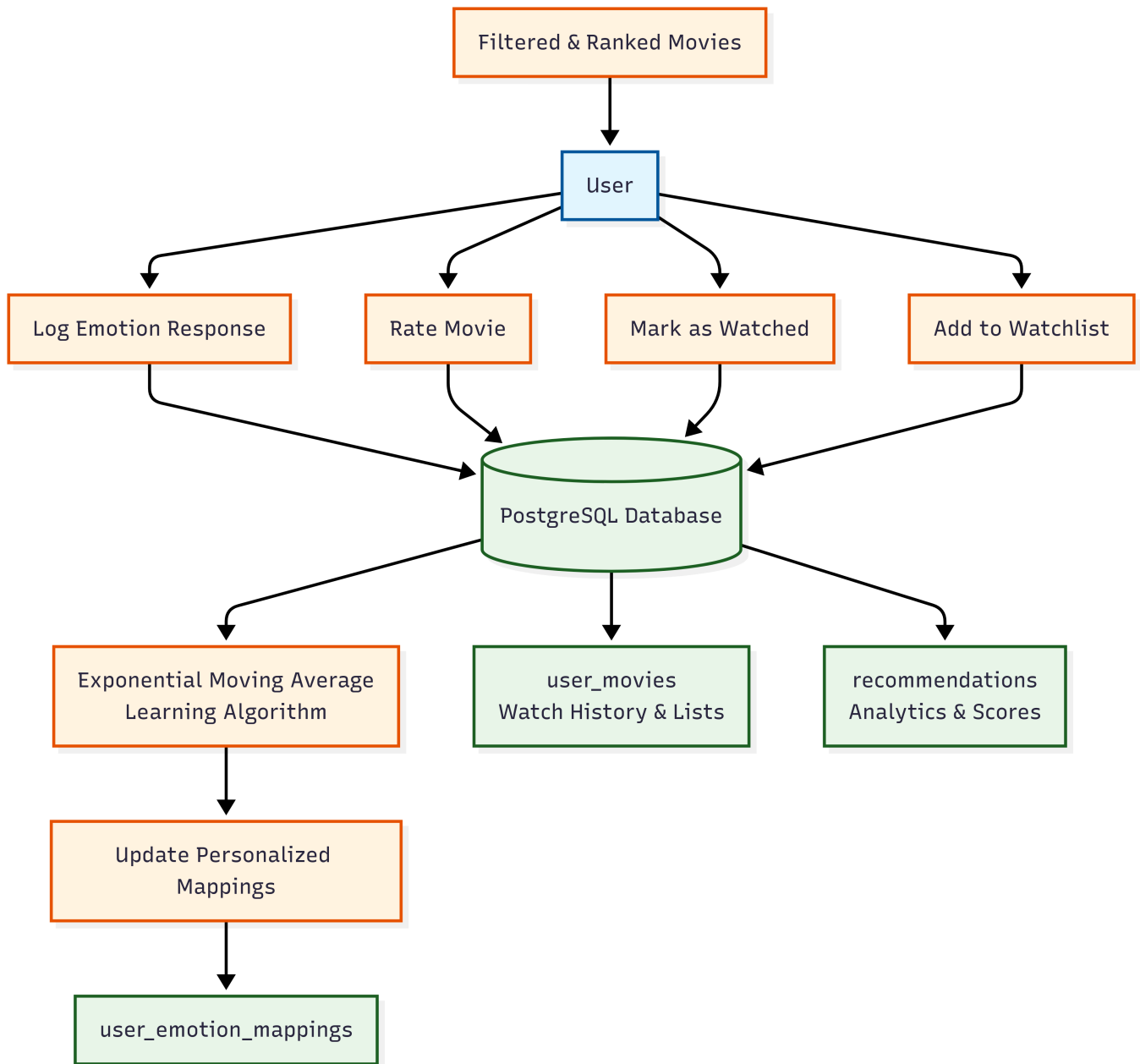
Recommendation Engine DFD



Emotion Mapping and User Management DFD



User Interaction DFD



2.1 Design Summary

EmotionFlix implements a modern three-tier architecture with clear separation of concerns:

Presentation Layer: React 19 frontend with TypeScript, providing responsive UI components for emotion detection, movie browsing, and user management.

Business Logic Layer: Node.js/Express REST API handling authentication, recommendation algorithms, and external service integration.

Data Layer: PostgreSQL database with optimized schema for emotion tracking, user preferences, and movie metadata caching.

2.2 Technical Stack

Frontend:

- **React 19** with TypeScript for type-safe component development

- **Vite** for fast development server and optimized production builds
- **Tailwind CSS v3.4** for utility-first styling and responsive design
- **React Router Dom v7** for client-side routing and navigation
- **Axios** for HTTP client requests to backend API
- **face-api.js v0.22** for client-side neural network emotion detection

Backend:

- **Node.js** with **Express v5** for RESTful API server
- **TypeScript** for type safety across the entire backend
- **PostgreSQL** for relational database with JSONB support
- **bcryptjs** for secure password hashing
- **jsonwebtoken** for JWT authentication
- **CORS** for cross-origin request handling

External Services:

- **TMDB API v3** for movie metadata, images, and search functionality

Development & Testing:

- **Frontend Testing:** Vitest with React Testing Library and jsdom environment
- **Backend Testing:** Jest with Supertest for API endpoint testing
- **Linting:** ESLint for code quality and consistency
- **Version Control:** Git with conventional commit practices

2.3 Key Architectural Decisions

2.3.1 Client-Side Emotion Detection

Decision: Implement emotion detection entirely on the client using face-api.js. **Rationale:** Ensures user privacy by avoiding server-side image processing and provides real-time feedback. **Implementation:** Pre-trained neural networks loaded locally with emotion enhancement algorithms.

2.3.2 Recommendation Algorithm

Decision: Emotion-based filtering **Rationale:** Provides more accurate recommendations by considering multiple factors: current emotion, historical emotional mapping data, and movie genres. **Implementation:** Multi-stage scoring system with configurable weights.

2.3.3 Progressive Data Collection

Decision: Allow anonymous usage with limited features, full functionality requires account creation.

Rationale: Reduces entry barriers while encouraging registration for personalized experiences.

Implementation: MovieMatch and basic browsing available anonymously, personalized recommendations require authentication.

3. Data Structures and File Organization

3.1 Database Architecture & Organization

The database implements a normalized relational model built on PostgreSQL, designed to support emotion tracking, personalized learning, and movie recommendation workflows. The schema prioritizes data integrity, query performance, and scalability.

3.1.1 Core Entity Design

User Management Layer:

- **users:** Handles authentication with secure password hashing, email validation, and account lifecycle tracking
- **user_emotion_profiles:** Stores current emotional states for quick access, updated as users interact with the system

Content & Metadata Layer:

- **movies:** Acts as TMDB data cache with complete movie metadata stored as JSONB for flexibility
- **genres:** Reference table containing TMDB's standard genre taxonomy (19 genres from Action to Western)
- **movie_genres:** Junction table implementing many-to-many relationships between movies and genres

Interaction Tracking Layer:

- **emotions:** Records every emotion detection session with method tracking (webcam/manual) and confidence scores
- **user_movies:** Manages user interactions (watchlist, watched status)
- **recommendations:** Analytics table storing recommendation scores

Personalization Layer:

- **user_emotion_mappings:** Core learning table storing personalized emotion-to-genre weights that evolve through user interactions
- Implements exponential moving averages for preference learning with configurable interaction weights

3.1.2 Data Relationships & Integrity

Cascading Delete Strategy: All user-related data implements **ON DELETE CASCADE** to ensure clean account deletion. Movie and genre data persist independently as reference material.

Unique Constraints:

- Email and username uniqueness for account management
- Composite keys prevent duplicate emotion sessions per user-movie interaction
- User-emotion-genre combinations ensure single weight per mapping

Data Types & Precision:

- Emotion scores stored as **DECIMAL(3,2)** for precise 0-1 range calculations

- Recommendation scores use `DECIMAL(5,4)` for granular ranking
- JSONB storage for flexible TMDb response caching with efficient querying

3.1.3 Performance Optimization

Strategic Indexing:

- User-based indexes for all personalization queries
- Movie quality indexes (vote_average, popularity) for recommendation filtering
- Composite indexes on frequently joined columns

Query Performance:

- Normalized structure reduces data duplication while maintaining join performance
- JSONB indexes enable fast movie metadata searches

3.2 TypeScript Data Models

3.2.1 Core Emotion Types

```
// Primary emotion scoring interface
interface EmotionScores {
  neutral: number; // 0-1 scale, normalized
  happy: number; // Joy, contentment, satisfaction
  sad: number; // Sorrow, melancholy, disappointment
  angry: number; // Frustration, irritation, rage
  fearful: number; // Anxiety, nervousness, apprehension
  disgusted: number; // Revulsion, distaste, aversion
  surprised: number; // Shock, amazement, astonishment
}

// Emotion detection session metadata
interface EmotionSession {
  id: string; // Unique session identifier
  type: 'webcam' | 'manual' | 'upload'; // Detection method
  emotionScores: EmotionScores; // Normalized emotion values
  confidence: number; // Detection confidence (0-1)
  timestamp: Date; // Session creation time
}

// Watched movie with emotion context
interface WatchedMovie {
  movieId: number; // TMDb movie identifier
  userId: string; // User identifier
  watchedAt: Date; // View timestamp
  emotions?: EmotionScores; // Associated emotional response
  hasLoggedEmotion: boolean; // Emotion data availability flag
  // Display metadata
  title: string;
  poster_path: string | null;
  release_date: string;
  vote_average: number;
```

```

genre_ids: number[];
}

```

3.2.2 Movie and Recommendation Types

```

// TMDB movie data structure
interface Movie {
  id: number;           // TMDB unique identifier
  title: string;         // Primary movie title
  overview: string;      // Plot synopsis
  release_date: string;  // Release date (YYYY-MM-DD)
  poster_path: string | null; // Poster image path
  backdrop_path: string | null; // Background image path
  genre_ids: number[];   // Associated genre identifiers
  genres?: Genre[];      // Full genre objects (detailed view)
  runtime?: number;      // Duration in minutes
  popularity: number;    // TMDB popularity score
  vote_average: number;  // Average user rating (0-10)
  vote_count: number;    // Total number of ratings
  adult: boolean;        // Adult content flag
  original_language: string; // Original language code
  original_title: string; // Original language title
  video: boolean;        // Video availability flag
  tagline?: string;      // Marketing tagline
}

// Recommendation scoring metadata
interface RecommendationScore {
  movieId: number;      // Movie being scored
  score: number;         // Calculated recommendation score
  reasons: string[];     // Human-readable scoring factors
}

// User preference profile
interface UserPreferences {
  favoriteGenres: number[]; // Preferred genre IDs
  emotionWeights: EmotionScores; // Personal emotion
  importance
  watchHistory: number[]; // Previously watched
  movies
  ratings: { [movieId: number]: number }; // User movie ratings
}

```

3.3 File Structure and Organization

3.3.1 Frontend Architecture


```
└─ emotionMapping.ts
└─ vite-env.d.ts
```

3.3.2 Backend Architecture

```
server/src/
├─ config/                # Configuration files
│   └─ database.ts
├─ controllers/           # Request handlers
│   ├── authController.ts
│   ├── emotionMappingController.ts
│   └─ userMoviesController.ts
├─ middleware/            # Express middleware
│   └─ auth.ts
├─ models/                # Data models
│   ├── User.ts
│   └─ UserEmotionMapping.ts
├─ routes/                # API route definitions
│   ├── auth.ts
│   ├── emotionMapping.ts
│   └─ userMovies.ts
├─ services/              # External integrations and business logic
└─ helpers
    └─ tmdbService.ts
└─ index.ts               # Server entry point
```

4. Classes and Models Implementation

4.1 Service Classes

4.1.1 EmotionDetectionService (**src/services/emotionDetection.ts**)

Purpose: Manages facial emotion detection using face-api.js neural networks for client-side analysis.

Key Methods:

- **LoadModels()**: Loads three neural network models (SSD MobileNet v1, Face Landmark 68, Face Expression Net)
- **DetectEmotionsFromImage()**: Analyzes static images for emotional content with confidence scoring
- **CaptureFromWebcam()**: Real-time webcam capture with stream management
- **EnhanceEmotionScores()**: Post-processes raw scores using amplification factors to boost subtle emotions
- **GetDominantEmotion()**: Identifies the strongest emotion from processed scores

Implementation Features:

- Client-side processing for privacy (no images sent to server)

- Model validation and retry logic for reliable loading
- Webcam stream cleanup to prevent memory leaks
- Aggressive emotion enhancement to detect subtle expressions

4.1.2 RecommendationService ([src/services/recommendationService.ts](#))

Purpose: Core recommendation engine combining emotion analysis with user preferences and movie metadata.

Key Methods:

- [getEmotionBasedRecommendations\(\)](#): Primary recommendation engine with emotion-to-genre mapping
- [filterAndRankMovies\(\)](#): Multi-factor scoring system for movie ranking
- [calculateRecommendationScore\(\)](#): Weighted scoring combining emotion compatibility, ratings, and movie popularity
- [calculateEmotionCompatibility\(\)](#): Computes compatibility between user emotions and movie genres

Implementation Features:

- Hybrid recommendation approach
- Integration with personalized emotion mapping service
- Support for both authenticated and anonymous users

4.1.3 PersonalizedEmotionMappingService ([src/services/personalizedEmotionMapping.ts](#))

Purpose: Dynamic emotion-to-genre mapping that learns from user behavior and adapts recommendations.

Key Methods:

- [getUserEmotionGenreMappings\(\)](#): Retrieves personalized mappings from database with caching
- [updateUserMappings\(\)](#): Updates mappings based on user interactions (ratings, watchlist additions)
- [getEnhancedDefaultMappings\(\)](#): Creates enhanced default mappings for new users
- [calculateGenrePreferences\(\)](#): Processes emotions to determine weighted genre preferences

Implementation Features:

- Exponential moving averages for learning from user behavior
- In-memory caching for performance
- Fallback to default mappings for new users

4.1.4 TMDBApiService ([src/services/tmdbApi.ts](#))

Purpose: Integration with The Movie Database API for movie data retrieval and caching.

Key Methods:

- [GetMoviesByGenres\(\)](#): Fetches movies by genre with pagination support

- `GetMovieDetails()`: Retrieves detailed movie information including runtime, cast, and reviews
- `SearchMovies()`: Text-based movie search with result filtering
- `GetPopularMovies()`: Trending and popular movie discovery
- `GetGenres()`: Retrieves available movie genres for mapping

Implementation Features:

- Axios-based HTTP client with API key authentication
- Response data transformation to consistent TypeScript interfaces
- Error handling for API rate limits and network issues

4.1.5 UserMoviesService (`src/services/userMoviesService.ts`)

Purpose: Manages user movie interactions including watchlist, watch history, and emotion data.

Key Methods:

- `getUserWatchlist()`: Retrieves user's saved movies with status filtering
- `addToWatchlist()`: Adds movies to user's watchlist with duplicate prevention
- `markAsWatched()`: Updates movie status and optionally records ratings
- `getUserWatchHistory()`: Fetches watch history with emotion data
- `logEmotionForMovie()`: Associates emotion data with watched movies

Implementation Features:

- CRUD operations for user-movie relationships
- Integration with emotion logging system

4.1.6 AuthService (`src/services/authService.ts`)

Purpose: Handles user authentication, registration, and session management.

Key Methods:

- `login()`: Authenticates users and returns JWT tokens
- `register()`: Creates new user accounts with validation
- `logout()`: Clears authentication tokens and session data
- `verifyToken()`: Validates JWT tokens and retrieves user data

Implementation Features:

- JWT token management with localStorage
- Form validation and error handling
- Integration with backend authentication endpoints

4.1.7 ApiClient (`src/services/apiClient.ts`)

Purpose: Centralized HTTP client for backend API communication with authentication and error handling.

Implementation Features:

- Axios instance with request/response interceptors

- Automatic JWT token attachment to authenticated requests

4.2 Data Models

4.2.1 User Model

Purpose: Represents user accounts with authentication and preference management.

Properties:

- Authentication: email, password hash, creation timestamps
- Preferences: favorite genres, emotion weights, viewing history

4.2.2 Movie Model

Purpose: Represents movie entities with TMDB integration and local caching.

Properties:

- Core data: title, overview, release date, ratings
- Media: poster and backdrop image paths
- Metadata: genres, runtime, popularity scores
- Caching: last updated timestamps, full TMDB responses

4.2.3 Emotion Model

Purpose: Represents user emotional sessions with detection metadata.

Properties:

- Emotion scores: normalized values for all seven emotions
- Detection metadata: method, confidence, timestamp

5. Core Features and Implementation

5.1 Emotion Detection System

5.1.1 Multi-Modal Input Support

- **Webcam Capture:** Real-time facial analysis with live preview
- **Image Upload:** Batch processing of user-provided photos
- **Manual Input:** Slider-based emotion entry with visual feedback
- **Confidence Scoring:** Quality assessment of detection accuracy

5.1.2 Neural Network Integration

- **Models:** SSD MobileNet v1, 68-point facial landmarks, expression recognition
- **Performance:** Client-side processing for privacy and speed
- **Enhancement:** Aggressive emotion amplification for subtle expression detection
- **Fallback:** Manual input when detection confidence is low

5.2 Recommendation Engine

5.2.1 Emotion-Based Filtering

- **Genre Mapping:** Dynamic emotion-to-genre correlation matrices
- **Personalization:** User-specific mapping refinement based on emotional profile

5.3 User Interface Design

5.3.1 Responsive Design System

- **Cross-Platform Compatibility:** Consistent experience across devices and browsers

5.4 Data Management

5.4.1 Privacy-First Architecture

- **Local Processing:** Emotion detection without server communication

5.4.2 Performance Optimization

- **Caching Strategy:** TMDB data cached locally
 - **Database Indexing:** Optimized queries for emotion and movie lookups
 - **Image Optimization:** Responsive images with appropriate sizing
-

6. Testing Strategy and Implementation

6.1 Testing Philosophy

The project implements a comprehensive testing strategy focusing on both security and functionality. The approach prioritizes real-world scenarios and production-ready validation ensuring the application handles edge cases and security threats effectively.

6.2 Testing Infrastructure

Frontend Testing:

- **Test Runner:** Vitest with React Testing Library
- **Environment:** jsdom for browser API simulation
- **Coverage:** 10 tests with 100% pass rate
- **Focus:** UI component rendering, user interactions, and client-side security

Backend Testing:

- **Test Runner:** Jest with Supertest for HTTP testing
- **Database:** Real PostgreSQL test database with automated schema setup
- **Coverage:** 59 tests with 100% pass rate
- **Focus:** API security, authentication, and database operations

6.3 Test Coverage Analysis

6.3.1 Backend Security Tests (59 Tests Total)

Authentication Security (19 tests):

- Registration with password hashing validation
- Login security with SQL injection prevention
- JWT token verification and error handling
- Input sanitization for malicious registration attempts

System Security (22 tests):

- SQL injection prevention across all endpoints
- Rate limiting and DoS protection
- Cross-user access prevention (horizontal privilege escalation)
- Password strength enforcement and secure storage
- Error message sanitization to prevent information leakage

Emotion Mapping Security (18 tests):

- CRUD operations with proper authorization
- Input validation for emotion and genre data
- XSS prevention in user data handling
- Database error handling with security focus

6.3.2 Frontend UI Tests (10 Tests Total)

Authentication Components (4 tests):

- AuthModal rendering and visibility controls
- LoginForm field validation and submission
- RegisterForm input handling and account creation
- Form state management

Emotion Detection Components (4 tests):

- EmotionCapture interface with multiple input methods
- EmotionDisplay percentage formatting and filtering
- ManualEmotionInput slider controls and real-time updates
- Component integration with emotion processing

Security UI Tests (2 tests):

- XSS prevention in form inputs
- Sensitive data handling (no console logging of passwords/tokens)

6.4 Test Execution Commands

```
# Frontend tests (Vitest)
npm test
npm run test:watch
```

```
npm run test:coverage

# Backend tests (Jest)
cd server && npm test
npm run test:security
```

7. Project Setup & Running

Prerequisites

- **Node.js** 18+ with npm
- **PostgreSQL** 17+ running locally or accessible remotely
- **TMDB API Key** - Sign up at themoviedb.org for free API access

Quick Start

1. Database Setup

```
# Start PostgreSQL (if using Docker)
docker compose up -d postgres

# Or use local PostgreSQL installation (it should be running on default
port 5432)
```

2. Environment Configuration

Create **.env** files in both root and server directories:

Root **.env**:

```
VITE_TMDB_API_KEY=your_tmdb_api_key_here
VITE_API_URL=http://localhost:3001/api
```

Server **.env**:

```
DATABASE_URL=postgresql://username:password@localhost:5432/emotionflix
JWT_SECRET=secure_jwt_secret
PORT=3001
NODE_ENV=development
```

3. Installation & Database Setup

```
# Install dependencies
npm install
cd server && npm install && cd ..

# Setup database schema
cd server
npm run db:setup # Creates database and applies schema
cd ..
```

4. Development Servers

```
# Terminal 1: Start backend server
cd server
npm run dev # Runs on http://localhost:3001

# Terminal 2: Start frontend development server
npm run dev # Runs on http://localhost:5173
```

Project Structure Overview

```
movie_rec/
├── src/           # Frontend React application
├── server/        # Backend Express API
├── database/      # Database schema and migrations
├── public/models/ # face-api.js neural network models
├── docs/          # Documentation
└── tests/         # Frontend test suites
```

Verification

- **Frontend:** Visit <http://localhost:5173> to access the application
- **Backend:** API health check at <http://localhost:3001/api/health>
- **Database:** Connect to verify tables were created successfully

A few things to note

Port Conflicts:

- Frontend runs on port 5173 (Vite default)
- Backend runs on port 3001 (configurable via PORT env var)
- PostgreSQL expects port 5432

TMDB API Setup:

- Free tier provides 1,000 requests per day
- API key required for movie data and images

Face-API Models:

- Models (~5MB total) load automatically on first emotion detection
 - Stored in `/public/models/` directory
-

8. Data Flow and Algorithm

8.1 System Architecture Overview

The system operates through five core components that work together to provide personalized movie suggestions based on users' emotional states.

8.2 Emotion Detection and Processing

8.2.1 Multi-Modal Emotion Input

The system supports three primary emotion detection methods:

- **Webcam Capture:** Real-time facial analysis using face-api.js neural networks
- **Image Upload:** Batch processing of user-provided photos
- **Manual Input:** Slider-based emotion entry with visual feedback

8.2.2 Neural Network Processing

Client-side emotion detection uses three neural network models:

- **SSD MobileNet v1:** Face detection and localization
- **68-Point Facial Landmarks:** Precise facial feature mapping
- **Expression Recognition:** Seven-emotion classification network

8.2.3 Emotion Enhancement Algorithm

Raw emotion scores undergo aggressive post-processing to improve sensitivity:

1. **Power Scaling:** Applies exponential scaling to reduce dominant emotion bias
2. **Amplification Factors:** Emotion-specific multipliers boost subtle expressions
3. **Threshold Filtering:** Removes insignificant emotion values below learned thresholds
4. **Normalization:** Ensures emotion scores sum to unity for probabilistic interpretation
5. **Diversity Boosting:** Additional amplification when multiple subtle emotions are detected

The enhancement process transforms typical neutral-dominant results into nuanced emotional profiles that better capture user states.

8.3 Personalized Emotion Mapping System

8.3.1 Default Mapping Initialization

New users receive default emotion-to-genre mappings with weighted preferences:

- **Happy:** Comedy, Family, Animation with decreasing weights

- **Sad:** Drama, Romance, War with genre-specific associations
- **Angry:** Action, Crime, Thriller for high-energy content
- **Fearful:** Horror, Thriller, Mystery for suspenseful experiences

8.3.2 Dynamic Learning Algorithm

User mappings evolve through exponential moving averages based on movie interactions.

8.3.3 Personalized Genre Recommendations

The system calculates weighted genre preferences by:

1. Processing each emotion intensity above threshold
2. Applying exponential weighting to amplify stronger emotions
3. Multiplying by personalized genre weights
4. Returning top-ranked genres for movie discovery

8.4 Recommendation Engine Architecture

8.4.1 Multi-Factor Scoring System

Movie recommendations use a weighted scoring formula combining multiple factors:

Popularity Factor Genre Preference Match TMDB Rating Filter Emotion Compatibility

8.4.2 Emotion Compatibility Calculation

For authenticated users, the system uses personalized mappings:

1. Iterates through user emotions above significance threshold
2. Calculates weighted compatibility for each movie genre
3. Applies emotion intensity × personalized genre weight scoring
4. Normalizes by total emotion weight for comparable scores

Anonymous users receive default static emotion-to-genre mapping.

8.5 Database Operations and Data Flow

8.5.1 User Registration and Profile Creation

New user registration triggers:

- User account creation with authentication credentials
- Empty emotion profile initialization
- Default emotion-to-genre mapping generation
- Personalized mapping storage in user_emotion_mappings table

8.5.2 Emotion Session Management

Each emotion detection session stores:

- Normalized emotion scores across seven categories
- Detection method and confidence metadata
- Session identifiers for tracking

8.5.3 Movie Log Processing

When users log new movies, the system:

1. Records associated emotions
2. Updates personalized emotion mappings
3. Stores movie in appropriate user lists (watchlist, watched)

8.5.4 User Preference Learning

User preferences evolve through interaction patterns:

- **Exponential Decay:** Recent interactions weighted more heavily in learning algorithms
 - **Genre Exploration:** System tracks user exploration of different content categories
-

9. Conclusion

9.1 Implementation Summary

This project demonstrates an emotion-based movie recommendation system. The implementation covers several key technical areas:

Machine Learning Integration: Successfully integrated face-api.js neural networks for client-side emotion detection, handling three different models and implementing custom emotion enhancement algorithms to improve detection sensitivity.

Full-Stack Architecture: Built a complete three-tier system with React frontend, Express API, and PostgreSQL database.

Complex Algorithm Design: Implemented a recommendation system that maps emotions to movie metadata. The personalized emotion mapping service demonstrates learning algorithms that adapt based on user emotion data.

Security and Testing: Developed a robust testing suite with 69 tests covering security vulnerabilities, API endpoints, and UI components. Implemented proper authentication, input validation, and protection against common security threats.

9.2 Problem Solving Approach

9.2.1 Technical Challenges Addressed

Building an emotion-based recommendation system presented several technical challenges:

Client-Side ML Processing: Implementing face-api.js required loading and managing neural network models in the browser, handling webcam streams properly, and processing images without sending data to servers for privacy.

Real-Time Recommendations: The system needed to process emotions and generates movie recommendations quickly enough for responsive user experience, requiring efficient API calls and caching strategies.

Database Design for Complex Relationships: The database schema had to handle multiple entity relationships (users, movies, emotions, preferences) while maintaining performance and data integrity.

9.2 Developer Experience and Learning Outcomes

9.2.1 What I Learned Building This Project

Building EmotionFlix was a significant learning experience that pushed me beyond typical web development projects into machine learning, complex algorithm design, and production-level testing.

Machine Learning in Practice: Working with face-api.js taught me how neural networks function in real applications, not just theory. I learned about model loading, performance optimization, and the challenges of processing data in real-time while maintaining user privacy.

Complex State Management: Managing emotional data, user preferences, movie information, and personalized mappings required careful architecture planning. I implemented caching strategies, learned about data normalization, and built systems that could handle multiple concurrent users.

Security: Writing 59 backend security tests taught me to think like an attacker. I learned to validate inputs, prevent SQL injection, handle authentication properly, and protect against common web vulnerabilities.

Algorithm Design: Creating the recommendation engine involved research into recommender systems, understanding how to weight different factors, and building learning systems that improve over time.

9.2.2 Technical Skills Developed

- **Full-Stack TypeScript:** Building applications from database to UI
- **Database Design:** Creating schemas with proper relationships and indexing
- **API Integration:** Working with external services (TMDB) while handling rate limits and errors
- **Testing:** Writing meaningful tests that cover both functionality and security
- **Performance Optimization:** Implementing caching, efficient queries, and client-side processing

9.2.3 Challenges

The most challenging aspect was making the emotion detection feel accurate and useful. Raw neural network outputs often showed high neutral values with subtle other emotions. I had to implement emotion enhancement algorithms that amplify meaningful emotional signals while maintaining realistic results.

Another significant challenge was building the personalized learning system. The lack of existing emotional data for movies presented a fundamental obstacle since there is no comprehensive database or dataset available that maps what movies make people feel specific emotions. This would require extensive data collection from users over time to ensure accuracy and build meaningful emotional profiles for different types of content.

Additionally, working with movie metadata presented its own set of constraints. Since I had to rely on free movie data sources, TMDB was essentially the only viable option that provided comprehensive movie

information. This limitation forced me to map emotions to genres rather than having access to more nuanced emotional content analysis with details about the plot, location, and themes. The API also imposed rate limits that required careful management, limiting the number of movies I could access within specific time periods and necessitating caching strategies.

9.3 Project Assessment

The development of EmotionFlix has been a valuable learning experience that extended beyond technical implementation into understanding real-world application potential. Working through the software development lifecycle taught me to consistently consider practical use cases, which shaped both the system architecture and user experience decisions.

With access to richer datasets, more sophisticated learning algorithms, and expanded content, I believe this application could realistically compete as a commercial application. While there remain areas for future enhancement—social features, additional content types, and more advanced machine learning models—the foundation I've established could scale to accommodate real-world usage. This project taught me that successful software development requires balancing technical implementation with user value and market viability, transforming what started as an academic project into a comprehensive exploration of product development principles.